Task 1: Implementation of MCP Neurons

↳ 1 cell hidden

MCP Neuron – AND Gate

```
def MCP_Neurons_AND(X1, X2, T):
    """
    Implements basic AND operations with MCP Neuron for two inputs.
    Arguments:
        X1 (list): An array of binary values.
        X2 (list): An array of binary values.
        T  (int) : Threshold value.
    Returns:
        state_neuron (list): State of neuron (1 or 0) for each input pair.
    """
    assert len(X1) == len(X2)
    sums = [X1[i] + X2[i] for i in range(len(X1))]

    state_neuron = [1 if s >= T else 0 for s in sums]
    return state_neuron

X1 = [0, 0, 1, 1]
X2 = [0, 1, 0, 1]
T = 2
result = MCP_Neurons_AND(X1, X2, T)
print(f"Output of AND gate for inputs {X1} and {X2} with threshold {T}: {result

print("\nAND Gate Truth Table:")
print(f"{'X1':>4} {'X2':>4} {'Output':>8}")
for x1, x2, out in zip(X1, X2, result):
    print(f"{x1:>4} {x2:>4} {out:>8}")
```

```
Output of AND gate for inputs [0, 0, 1, 1] and [0, 1, 0, 1] with threshold 2: [0

AND Gate Truth Table:
  X1   X2   Output
   0    0        0
   0    1        0
   1    0        0
   1    1        1
```

MCP Neuron – OR Gate

```
def MCP_Neurons_OR(X1, X2, T):
    """
    Implements basic OR operations with MCP Neuron for two inputs.
    Arguments:
        X1 (list): An array of binary values.
        X2 (list): An array of binary values.
        T  (int) : Threshold value.
    Returns:
        state_neuron (list): State of neuron (1 or 0) for each input pair.
    """
    assert len(X1) == len(X2)

    sums = [X1[i] + X2[i] for i in range(len(X1))]

    state_neuron = [1 if s >= T else 0 for s in sums]
    return state_neuron

X1 = [0, 0, 1, 1]
X2 = [0, 1, 0, 1]
T = 1
result_or = MCP_Neurons_OR(X1, X2, T)
print(f"Output of OR gate for inputs {X1} and {X2} with threshold {T}: {result_

print("\nOR Gate Truth Table:")
print(f"{'X1':>4} {'X2':>4} {'Output':>8}")
for x1, x2, out in zip(X1, X2, result_or):
    print(f"{x1:>4} {x2:>4} {out:>8}")
```

```
Output of OR gate for inputs [0, 0, 1, 1] and [0, 1, 0, 1] with threshold 1: [0,

OR Gate Truth Table:
  X1   X2   Output
   0    0        0
   0    1        1
   1    0        1
   1    1        1
```

Question- 1: List out all the limitations of MCP- Neurons.**

The MCP Neuron has several key limitations:

1. **Only handles binary inputs and outputs** - MCP neurons accept only 0 or 1 as inputs. They cannot process continuous or real-valued data.
2. **No learning mechanism** - The threshold T is hand-crafted manually. The MCP neuron has no ability to learn from data and adjust its parameters.
3. **No weight differentiation (no measure of importance)** - All inputs are treated equally. There is no concept of weighted inputs, so it cannot reflect that some

features matter more.

4. **Cannot solve non-linearly separable problems** - A single MCP neuron can only represent linearly separable Boolean functions (AND, OR). It fails on XOR.
5. **Represents only a single neuron** - It models one neuron in isolation and cannot capture complex connectivity patterns.
6. **Highly simplified biological model** - It is an extreme simplification. We cannot draw conclusions about real neural behaviour from it.
7. **Inhibitory inputs are hard-coded** - Forcing output to 0 on any inhibitory input is a rigid rule, not a learned behaviour.

Question- 2: Think if you can develop a logic to solve for XOR function using MCP Neuron. {Can you devise a if else rules.}

A single MCP neuron cannot solve XOR because XOR is not linearly separable. You would need a multi-layer structure (eg, combining AND, OR, and NOT gates).

```
def MCP_Neuron_generic(inputs, T):
    """Generic MCP Neuron (no inhibitory inputs)."""
    return 1 if sum(inputs) >= T else 0

def MCP_XOR(x1, x2):
    """XOR using two-layer MCP neurons: AND( OR(x1,x2), NAND(x1,x2) )"""
    or_out   = MCP_Neuron_generic([x1, x2], T=1)
    and_out  = MCP_Neuron_generic([x1, x2], T=2)
    nand_out = 1 - and_out
    xor_out  = MCP_Neuron_generic([or_out, nand_out], T=2)
    return xor_out

print("XOR using Two-Layer MCP Network:")
print(f"{'X1':>4} {'X2':>4} {'XOR':>6}")
for x1, x2 in [(0,0),(0,1),(1,0),(1,1)]:
    print(f"{x1:>4} {x2:>4} {MCP_XOR(x1,x2):>6}")
print("\nConclusion: A single MCP neuron cannot solve XOR (not linearly separat
print("But by chaining two MCP neurons (OR + NAND -> AND), XOR can be realised.
```

```
XOR using Two-Layer MCP Network:
  X1   X2    XOR
   0    0      0
   0    1      1
   1    0      1
   1    1      0

Conclusion: A single MCP neuron cannot solve XOR (not linearly separable).
But by chaining two MCP neurons (OR + NAND -> AND), XOR can be realised.
```

Task 2: Perceptron Algorithm for 0 vs 1 Classification

```
df_0_1 = pd.read_csv("/content/drive/MyDrive/aayushma/mnist_0_and_1.csv")

X = df_0_1.drop(columns=["label"]).values
y = df_0_1["label"].values

print("Feature matrix shape:", X.shape)
print("Label vector shape:", y.shape)
```

```
Feature matrix shape: (12665, 784)
Label vector shape: (12665,)
```

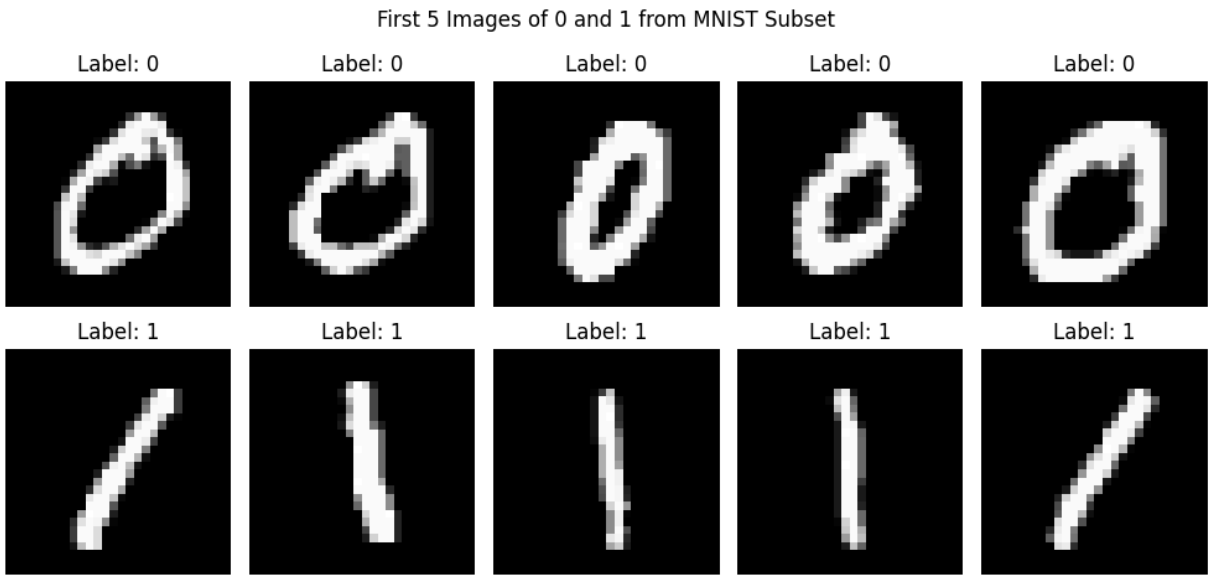
What does the shape of X represent?

It represents the number of samples and the number of features (pixels) per sample. Shape of X: `(n_samples, 784)`

What does the shape of Y represent?

It represents a 1D vector containing the true labels (target values) for every sample in the dataset. Shape of y: `(n_samples,)`

```
images_0 = X[y == 0]
images_1 = X[y == 1]

fig, axes = plt.subplots(2, 5, figsize=(10, 5))
if len(images_0) < 5 or len(images_1) < 5:
    print("Error: Not enough images to plot 5 images.")
else:
    for i in range(5):
        axes[0, i].imshow(images_0[i].reshape(28, 28), cmap="gray")
        axes[0, i].set_title("Label: 0"); axes[0, i].axis("off")
        axes[1, i].imshow(images_1[i].reshape(28, 28), cmap="gray")
        axes[1, i].set_title("Label: 1"); axes[1, i].axis("off")
plt.suptitle("First 5 Images of 0 and 1 from MNIST Subset")
plt.tight_layout(); plt.show()
```




```
weights = np.zeros(X.shape[1])
bias = 0
learning_rate = 0.1
epochs = 100
```

What does the weights array represent in this context?

In this context, the weights array represents the importance or "strength" assigned to each of the 784 pixels. During training, these weights are adjusted to determine which pixels are most characteristic of a "0" versus a "1"

Why are we initializing the weights to zero? What effect could this have on the training process?

Initializing weights to zero provides a neutral starting point where no pixel is initially considered more important than another.

```
def decision_function(X, weights, bias):
    """
    Compute predicted labels using a step activation function.
    Returns y_pred_all: 1 if z >= 0, else 0.
    """
    predictions = np.dot(X, weights) + bias
    y_pred_all = np.where(predictions >= 0, 1, 0)
    return y_pred_all
```

```
def train_perceptron(X, y, weights, bias, learning_rate=0.1, epochs=100):
    """
    Train the perceptron using the Perceptron Learning Algorithm.
    Returns: weights, bias, accuracy
    """
    for epoch in range(epochs):
        convergence_flag = True
        for i in range(X.shape[0]):

            output = np.dot(X[i], weights) + bias

            y_pred = 1 if output >= 0 else 0

            if y_pred != y[i]:
                error = y[i] - y_pred
                weights = weights + learning_rate * error * X[i]
                bias    = bias    + learning_rate * error
                convergence_flag = False
        if convergence_flag:
            print(f"Converged at epoch {epoch + 1}")
            break
    y_pred_all = decision_function(X, weights, bias)
    accuracy = np.mean(y_pred_all == y)
    return weights, bias, accuracy
```

What is the purpose of the output = np.dot(X[i], weights) + bias line?

This line calculates the net weighted input (z). It aggregates all input features by multiplying them by their respective weights and adding the bias to determine how far the current sample is from the decision boundary

What happens when the prediction is wrong? How are the weights and bias updated?

If the prediction is wrong, the learning algorithm updates the weights and bias to reduce the error. Weights are updated using the formula $w_j = w_j + \eta(y - \hat{y})x_j$, and the bias

is updated by $b = b + \eta(y - \hat{y})$.

Why is the final accuracy important, and what do you expect it to be?

Final accuracy is important because it measures how well the perceptron has learned to separate the two classes. For 0 vs 1, I expect near 100% accuracy because these digits are visually distinct and linearly separable.

```
weights = np.zeros(X.shape[1])
bias = 0
weights, bias, accuracy = train_perceptron(X, y, weights, bias)
print(f"The Final Accuracy is: {accuracy:.4f}")
```

```
Converged at epoch 13
The Final Accuracy is: 1.0000
```

```
predictions = np.dot(X, weights) + bias
y_pred = np.where(predictions >= 0, 1, 0)
final_accuracy = np.mean(y_pred == y)
print(f"Final Accuracy: {final_accuracy:.4f}")

misclassified_idx = np.where(y_pred != y)[0]
if len(misclassified_idx) > 0:
    fig, axes = plt.subplots(2, 5, figsize=(10, 5))
    for ax, idx in zip(axes.flat, misclassified_idx[:10]):
        ax.imshow(X[idx].reshape(28, 28), cmap="gray")
        ax.set_title(f"Pred: {y_pred[idx]}, True: {y[idx]}")
        ax.axis("off")
    plt.suptitle("Misclassified Images (0 vs 1)")
    plt.tight_layout(); plt.show()
else:
    print("All images were correctly classified!")
```

```
Final Accuracy: 1.0000
All images were correctly classified!
```

What does misclassified idx store, and how is it used in this code?

This variable stores the indices (row numbers) of all samples where the predicted label did not match the true label. It is used to filter the dataset so that only the failed images are displayed for visual analysis.

How do you interpret the result if the output is "All images were correctly classified!"?

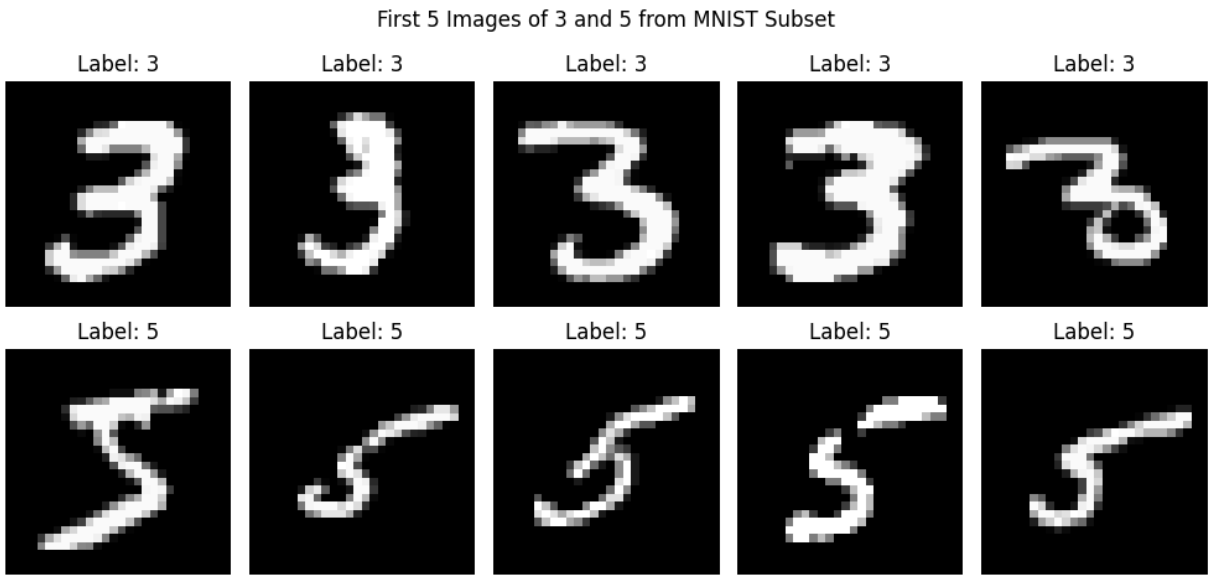
This result indicates that the Perceptron has reached convergence. It means the dataset is linearly separable, and the model found a perfect decision boundary that separates all samples of one class from the other.

Task 3: Perceptron Algorithm for 3 vs 5 Classification

```
df_3_5 = pd.read_csv("/content/drive/MyDrive/aayushma/mnist_3_and_5.csv")
X_35 = df_3_5.drop(columns=["label"]).values
y_35 = df_3_5["label"].values

y_35_binary = np.where(y_35 == 3, 0, 1)

print("Feature matrix shape:", X_35.shape)
print("Class distribution:", dict(zip(*np.unique(y_35, return_counts=True))))
```

```
Feature matrix shape: (2741, 784)
Class distribution: {np.int64(3): np.int64(1484), np.int64(5): np.int64(1257)}
```

```
images_3 = X_35[y_35 == 3]
images_5 = X_35[y_35 == 5]
fig, axes = plt.subplots(2, 5, figsize=(10, 5))
for i in range(5):
    axes[0, i].imshow(images_3[i].reshape(28, 28), cmap="gray")
    axes[0, i].set_title("Label: 3"); axes[0, i].axis("off")
    axes[1, i].imshow(images_5[i].reshape(28, 28), cmap="gray")
    axes[1, i].set_title("Label: 5"); axes[1, i].axis("off")
plt.suptitle("First 5 Images of 3 and 5 from MNIST Subset")
plt.tight_layout(); plt.show()
```



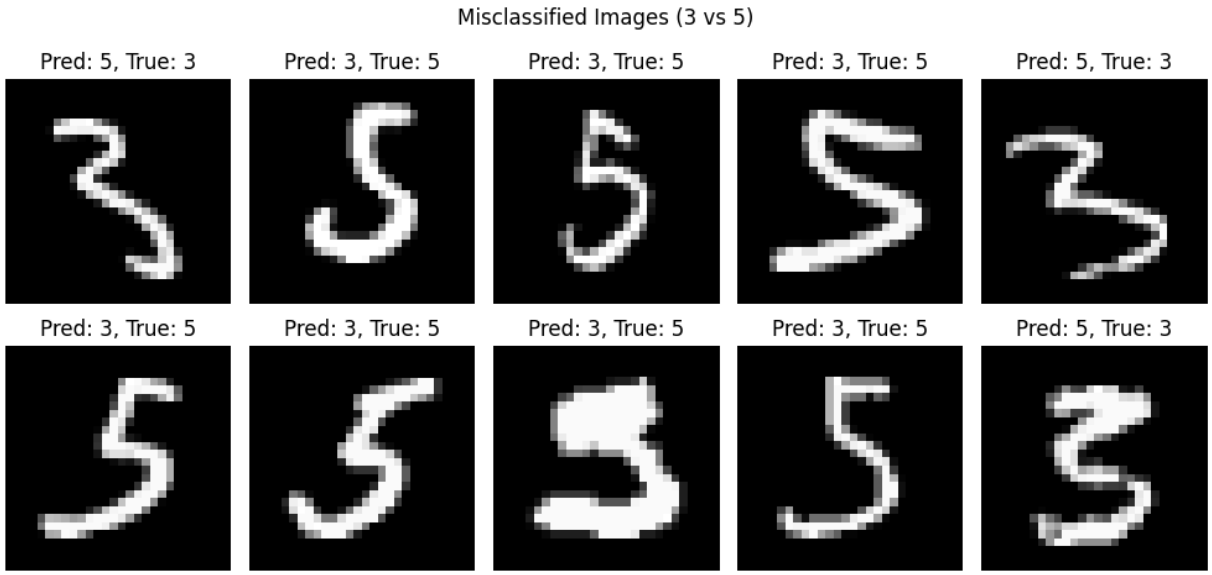

```
weights_35 = np.zeros(X_35.shape[1])
bias_35 = 0
weights_35, bias_35, accuracy_35 = train_perceptron(
    X_35, y_35_binary, weights_35, bias_35, learning_rate=0.1, epochs=100
)
print(f"Final Training Accuracy (3 vs 5): {accuracy_35:.4f}")
```

```
Final Training Accuracy (3 vs 5): 0.9869
```

```
predictions_35 = np.dot(X_35, weights_35) + bias_35
y_pred_35 = np.where(predictions_35 >= 0, 1, 0)
y_pred_labels = np.where(y_pred_35 == 0, 3, 5)

final_accuracy_35 = np.mean(y_pred_35 == y_35_binary)
print(f"Final Accuracy (3 vs 5): {final_accuracy_35:.4f}\n")

misclassified_35 = np.where(y_pred_35 != y_35_binary)[0]
if len(misclassified_35) > 0:
    fig, axes = plt.subplots(2, 5, figsize=(10, 5))
    for ax, idx in zip(axes.flat, misclassified_35[:10]):
        ax.imshow(X_35[idx].reshape(28, 28), cmap="gray")
        ax.set_title(f"Pred: {y_pred_labels[idx]}, True: {y_35[idx]}")
        ax.axis("off")
    plt.suptitle("Misclassified Images (3 vs 5)")
```

```
    plt.tight_layout(); plt.show()

    print(f"\nTotal misclassified: {len(misclassified_35)} / {len(y_35)}")
else:
    print("All images were correctly classified!")
```

```
Final Accuracy (3 vs 5): 0.9869
```




```
Total misclassified: 36 / 2741
```